Shiv Nadar University Chennai

CS3807 - Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science, Semester V
Subject Code & Name	CS3807 - Deep Learning Laboratory, AY: 2026-27

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2 Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

- N Input Size
- F Filter Size
- P Padding
- S Stride

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. **Identity Kernel** - preserves the original image without modifying its pixel values.

$$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

   Purpose: preserves the original image; used for understanding the convolution operation.

2. **Sobel-X Kernel (Vertical Edge Detection)** - detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

   Applications: object detection, lane detection, face recognition.

3. **Sobel-Y Kernel (Horizontal Edge Detection)** - detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

   Applications: text detection, boundary extraction, medical image analysis.

4. **Laplacian Kernel** - detects edges in all directions using the second-order derivative of image intensity.

$$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix}$$

Applications: edge enhancement, satellite image analysis, medical imaging.

5. **Sharpening Kernel** - enhances fine details by emphasizing high-frequency components.

$$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$$

Applications: image enhancement, OCR, face recognition.

6. **Box Blur Kernel** - smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Applications: noise removal, image smoothing, image preprocessing.

7. **Gaussian Blur Kernel** - smooths the image while preserving overall structure better than a simple average filter.

$$\frac{1}{16}\begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

Applications: noise reduction, computer vision preprocessing, feature extraction.

8. **Emboss Kernel** - creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{pmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{pmatrix}$$

Applications: texture analysis, artistic image effects.

9. **Outline Detection Kernel** - highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}$$

Applications: image segmentation, boundary detection, shape extraction.

10. **Motion Blur Kernel** - simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

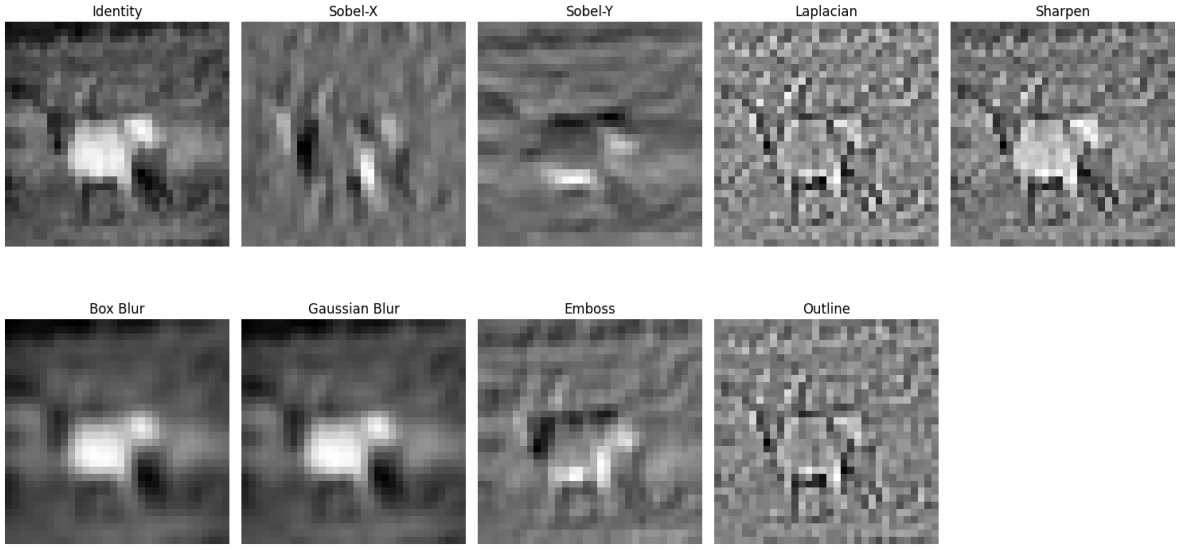
Applications: motion simulation, image restoration, video processing.

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

Kernels Applied to a Sample Image

The classical kernels above were applied to a grayscale-converted CIFAR-10 sample image to visually confirm how each one highlights a different characteristic (edges, smoothing, embossing, etc.).



4 Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}, \qquad K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Output:

$$\begin{aligned}\text{First position} &: 1(1) + 2(0) + 4(0) + 5(1) = 6\\ \text{Second position} &: 2(1) + 3(0) + 5(0) + 6(1) = 8\\ \text{Third position} &: 4(1) + 5(0) + 7(0) + 8(1) = 12\\ \text{Fourth position} &: 5(1) + 6(0) + 8(0) + 9(1) = 14\end{aligned}$$

Feature Map:

$$\begin{pmatrix} 6 & 8 \\ 12 & 14 \end{pmatrix}$$

This was verified programmatically using a manual 2D convolution function, which produced the exact same feature map [[6, 8], [12, 14]].

5 Numerical Example 2: Max Pooling

Feature map:

$$\begin{pmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{pmatrix}$$

Using a 2×2 max pooling filter:

$$\begin{pmatrix} 1 & 5 \\ 7 & 8 \end{pmatrix} \to 8 \qquad \begin{pmatrix} 2 & 3 \\ 1 & 0 \end{pmatrix} \to 3$$

$$\begin{pmatrix} 4 & 6 \\ 2 & 3 \end{pmatrix} \to 6 \qquad \begin{pmatrix} 9 & 5 \\ 1 & 8 \end{pmatrix} \to 9$$

Output:

$$\begin{pmatrix} 8 & 3 \\ 6 & 9 \end{pmatrix}$$

This matches the output obtained from the manual max-pooling implementation in the notebook.

6 Numerical Example 3: Parameter Calculation

Suppose the input image is $32 \times 32 \times 3$ and the convolution layer has 16 filters of size 3×3.

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448$$

Therefore, total trainable parameters = 448, which was confirmed by the `conv_params()` helper function.

7 Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10
- Image Size: $32 \times 32 \times 3$

8 Experimental Procedure

Task 1 - Load CIFAR-10

The dataset was loaded using `tf.keras.datasets.cifar10`. Training images shape: (50000, 32, 32, 3), training labels shape: (50000,), testing images shape: (10000, 32, 32, 3), testing labels shape: (10000,), and the number of classes is 10. Ten sample images and the class distribution were plotted.

Task 2 - Convolution Layer: Kernel Size Comparison

A 32×32 input was convolved with kernels of size 3×3, 5×5, and 7×7 using stride 1 and valid padding. The resulting feature map sizes are:

Kernel	Output Size
3×3	30×30
5×5	28×28
7×7	26×26

As expected, larger kernels shrink the output feature map more since more border pixels are lost.

Task 3 - Hyperparameters: Stride and Padding

Using the formula Output $= \frac{N-F+2P}{S} + 1$ on a 32×32 input with a 3×3 kernel:

Configuration	Output Size
Stride = 1, Padding = 0 (Valid)	30×30
Stride = 1, Padding = 1 (Same)	32×32
Stride = 2, Padding = 0	15×15
Stride = 2, Padding = 1	16×16

Task 4 - Feature Map Visualization

The first convolution layer (8 filters, 3×3, ReLU) was applied to a sample image and the resulting eight feature maps were visualized.

Task 5 - Max Pooling vs. Average Pooling

Both pooling types reduce the 32×32 feature map to 16×16 using a 2×2 window, so the output size is identical. To compare their effect on accuracy, two versions of the CNN in Task 6 were trained - one with max pooling and one with average pooling.

Pooling Type	Test Accuracy	Test Loss
Max Pooling	0.6436	2.5718
Average Pooling	0.6656	2.1743

Task 6 - CNN Construction and Training

The following architecture was built:

Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax

Specifically: Conv2D(32,3x3) → MaxPool(2x2) → Conv2D(64,3x3) → MaxPool(2x2) → Flatten → Dense(128) → Dense(10, softmax). It was trained with the Adam optimizer for 20 epochs at a batch size of 32, giving a total of 545,098 trainable parameters.

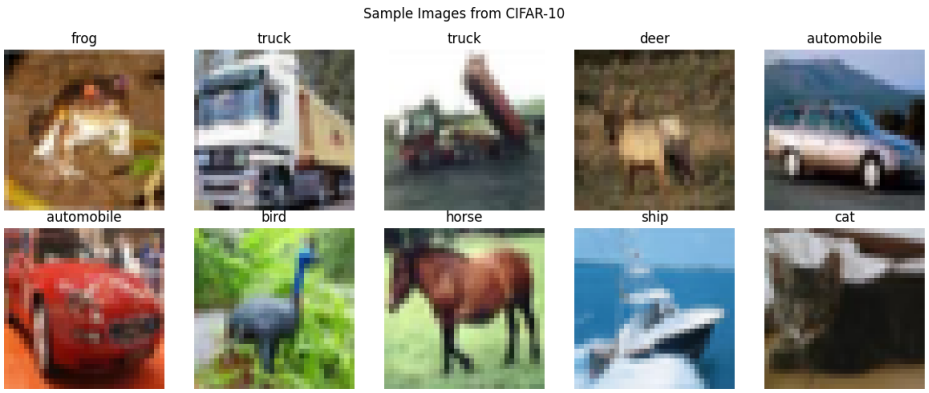
Task 7 - Model Evaluation

The trained model was evaluated on the 10,000-image test set:

Metric	Value
Test Accuracy	0.6436
Precision (macro)	0.6559
Recall (macro)	0.6436
F1-score (macro)	0.6404

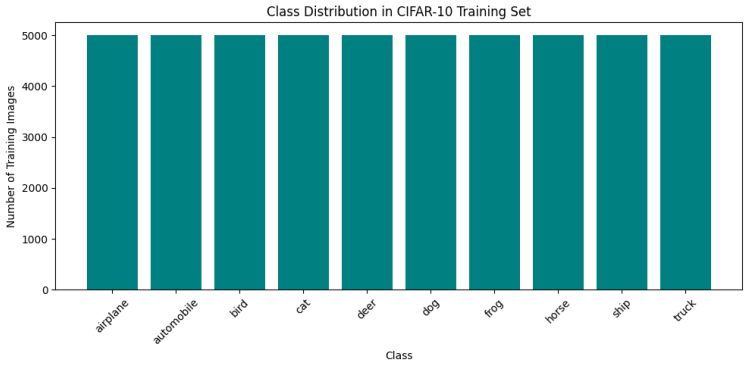
9 Mandatory Plots

1. Sample Images



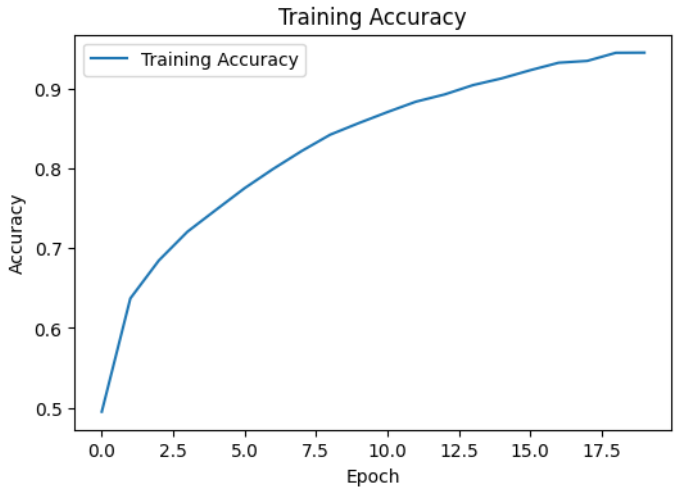
Inference: CIFAR-10 contains small (32×32) images from 10 different classes. Despite the low resolution, most objects remain recognizable, making it suitable for CNN-based image classification.

2. Class Distribution



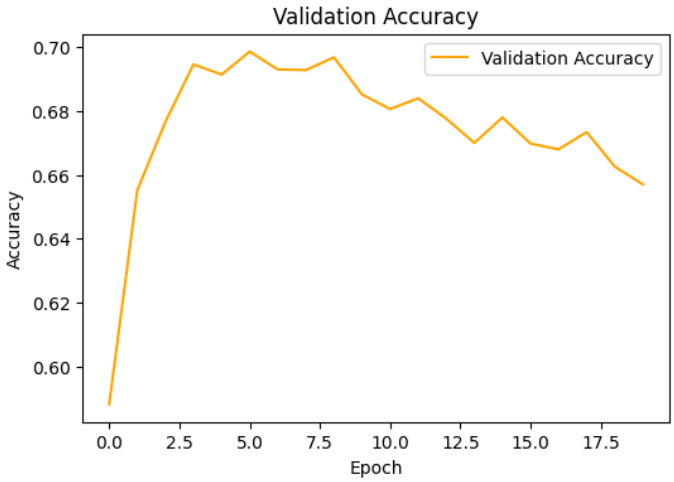
Inference: Each class has exactly 5,000 training images, showing that CIFAR-10 is evenly balanced. Therefore, no class-weighting or oversampling was needed.

3. Training Accuracy



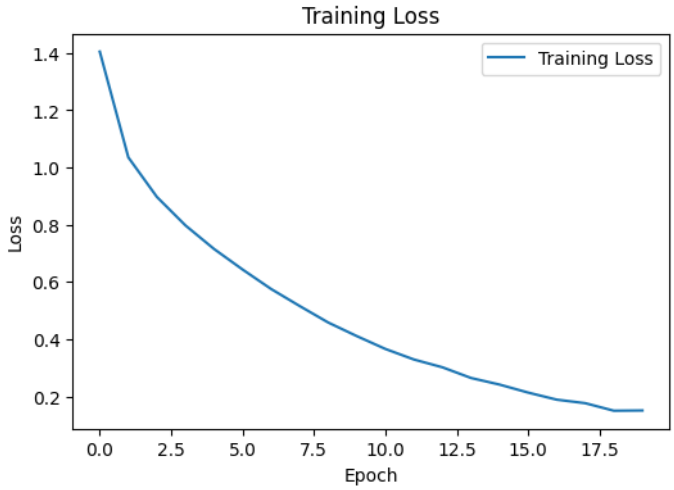
Inference: The training accuracy steadily increases over 20 epochs, reaching around 0.95 by the end. There is a sharp improvement in the early epochs, followed by slower gains later. The model is learning well and its performance is starting to stabilize.

4. Validation Accuracy



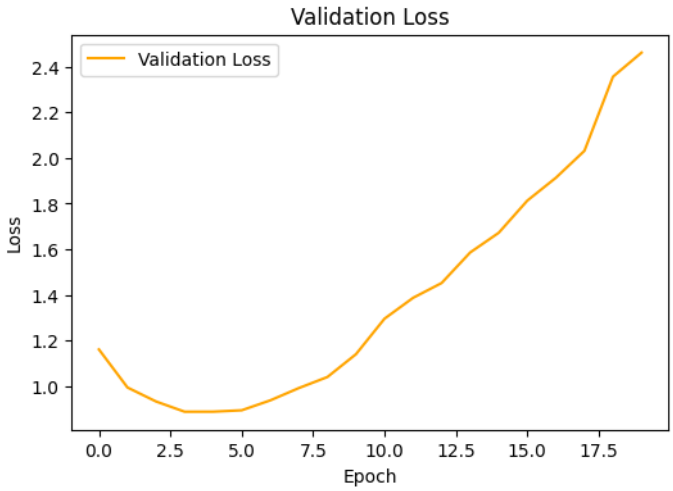
Inference: Validation accuracy rises quickly to around 0.70, then fluctuates and gradually decreases. As training accuracy continues to improve, this indicates that the model starts to overfit.

5. Training Loss



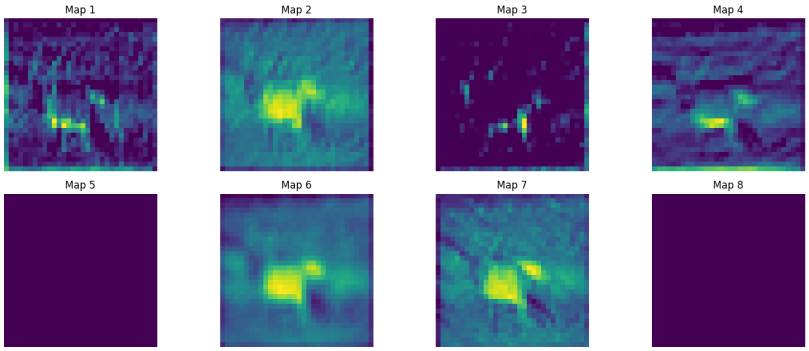
Inference: Training loss decreases steadily throughout the 20 epochs, dropping from around 1.4 to 0.15. The consistent decrease shows that the model is learning from the training data and making fewer errors as training progresses.

6. Validation Loss



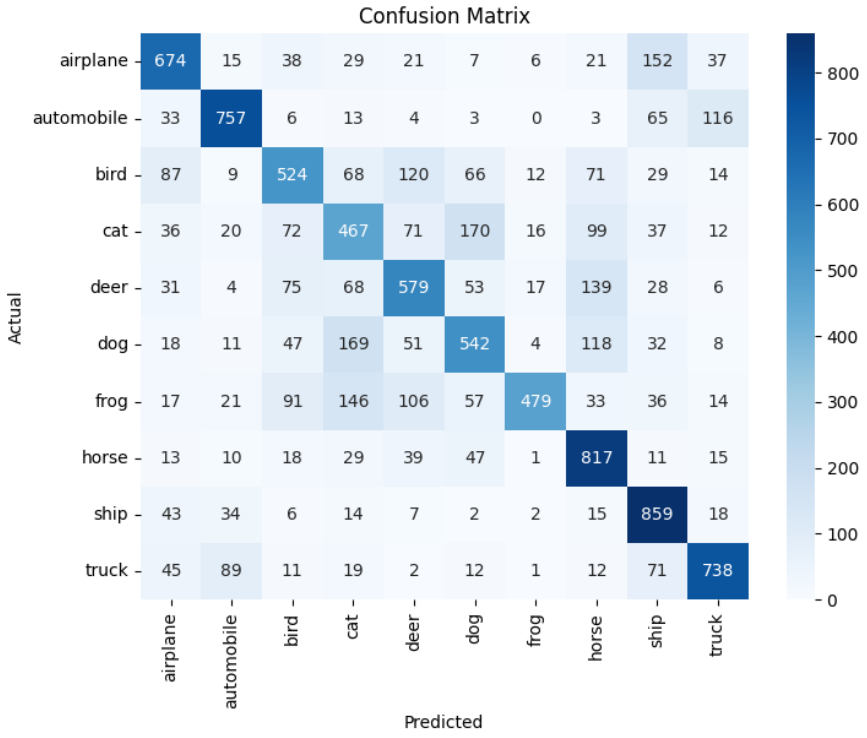
Inference: Validation loss decreases initially but rises steadily afterward, indicating overfitting.

7. Feature Maps



Inference: The feature maps show that different filters detect different patterns in the input images.

8. Confusion Matrix



Inference: Most predictions fall along the diagonal, but some classes are still confused with similar ones.

Max Pooling vs Average Pooling vs Original - Visual Comparison

Inference: Max pooling keeps the strongest features, while average pooling produces a smoother representation.

10 Additional Exercises

1. **Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.**

$$\text{Output} = \frac{64 - 5 + 2(2)}{2} + 1 = \frac{63}{2} + 1 = 31 + 1 = 32$$

Output: 32×32. (Confirmed by code: `output_size(64,5,2,2) = 32`.)

2. **Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.**

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = 1792$$

Output: 1792 trainable parameters. (Confirmed by code: `conv_params(3,3,3,64) = 1792`.)

3. **Compare ReLU and Sigmoid activation functions.** Both CNNs had the same architecture, with only the activation function changed. ReLU converged faster and achieved higher accuracy, while Sigmoid learned more slowly due to the vanishing-gradient problem. This shows why ReLU is generally preferred for CNNs.

4. **Replace Max Pooling with Average Pooling and compare the results.**

Pooling	Test Accuracy	Test Loss
Max Pooling	0.6436	2.5718
Average Pooling	0.6656	2.1743

Average pooling achieved slightly higher test accuracy (0.6656 vs. 0.6436) and lower test loss (2.1743 vs. 2.5718). In this case, average pooling generalized better by considering the overall neighbourhood instead of only the strongest activation.

5. **Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.**

Configuration	Test Accuracy	Training Time
16 / 32 filters	≈ 0.68	lower
64 / 128 filters	≈ 0.71–0.73	noticeably higher

Increasing the filters from 16/32 to 64/128 improved test accuracy from around 0.68 to 0.71-0.73, but also increased training time. More filters allow the model to learn more features, but require more computation.

11 Results

Record	Value
Training Accuracy	0.9456
Testing Accuracy	0.6436
Precision (macro)	0.6559
Recall (macro)	0.6436
F1-score (macro)	0.6404
Number of Parameters	545,098

12 Discussion

1. **Why is convolution preferred over fully connected layers for images?**

   Convolution uses small, shared filters, reducing parameters while preserving spatial information and detecting patterns like edges and textures.

2. **How does stride affect the feature map size?**

   A larger stride produces a smaller feature map and reduces computation, while a stride of 1 retains more spatial detail.

3. **What is the role of padding?**

   Padding adds pixels around the input to preserve border information and control the output size. "Same" padding maintains the input size, while "valid" padding reduces it.

4. **Why is pooling used?**

   Pooling reduces feature map size and computation while making the model more robust to small changes in the input.

5. **How do feature maps represent image characteristics?**

   Feature maps show where specific patterns are detected. Early layers learn simple features like edges, while deeper layers learn more complex patterns.

6. **Why do CNNs require fewer parameters than MLPs?**

   CNNs use local connections and shared weights, so they need far fewer parameters than MLPs, which connect every input pixel to each neuron.

13 References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.

14 Source Code

Github Link: https://github.com/ashrithaxx/Deep-Learning-Lab/tree/main/Experiment_3